

Ticket Intelligence

Streaming ELT and semantic search over 28,587 support tickets

Kafka · Spark Structured Streaming · MinIO · Elasticsearch

AI capability: embeddings + semantic search, RAG, k-NN routing

The problem, and why this dataset

Customer IT Support Ticket Dataset — Tobias Bueck, CC BY-NC 4.0

Two costs in every support organisation

- Someone must route each free-text ticket to the right queue.
- Agents re-solve problems the organisation already solved.

Why it qualifies as semi-structured

- subject / body / answer are natural language, averaging 387 characters.
- tag_1...tag_8 is a ragged list of 1,255 values flattened into sparse columns — 98% null by tag_8.
- subject itself is null in 13.4% of rows. That is data, not an error.

28,587

tickets

en 57% / de 43%

10

queues

the routing label

4

ticket types

Incident · Request
Problem · Change

1,255

distinct tags

ragged, sparse

Majority class is Technical Support at 29.3% — the number every model must beat.

Architecture

ELT, not ETL — the broker holds raw truth and Spark does the transformation



Enrichment applied to every ticket

normalise text · redact PII · detect language · sentiment · rule classify · embed 384-d

All of it is pure Python in core/ — Spark calls the same functions the unit tests call.

Why foreachBatch instead of three streaming sinks? One micro-batch fans out to two Parquet paths and Elasticsearch. Three separate writeStream queries would re-run the expensive enrichment for each sink.

Why a content-hash ticket_id? Kafka gives at-least-once. Hashing content makes every sink idempotent, so a replayed batch overwrites rather than duplicates — effectively-once without transactions.

The AI capability

Four of the brief's options; embeddings + semantic search is the centrepiece

b

Embeddings + semantic search

384-d vectors in an Elasticsearch dense_vector field, cosine kNN

f

k-NN routing classifier

Reuses the same vectors — no separate model, no training run

c

RAG question answering

Retrieves k tickets, answers only from them, citations verified

a

LLM enrichment

Classify, tag, summarise — output validated against known labels

Two backends, one interface

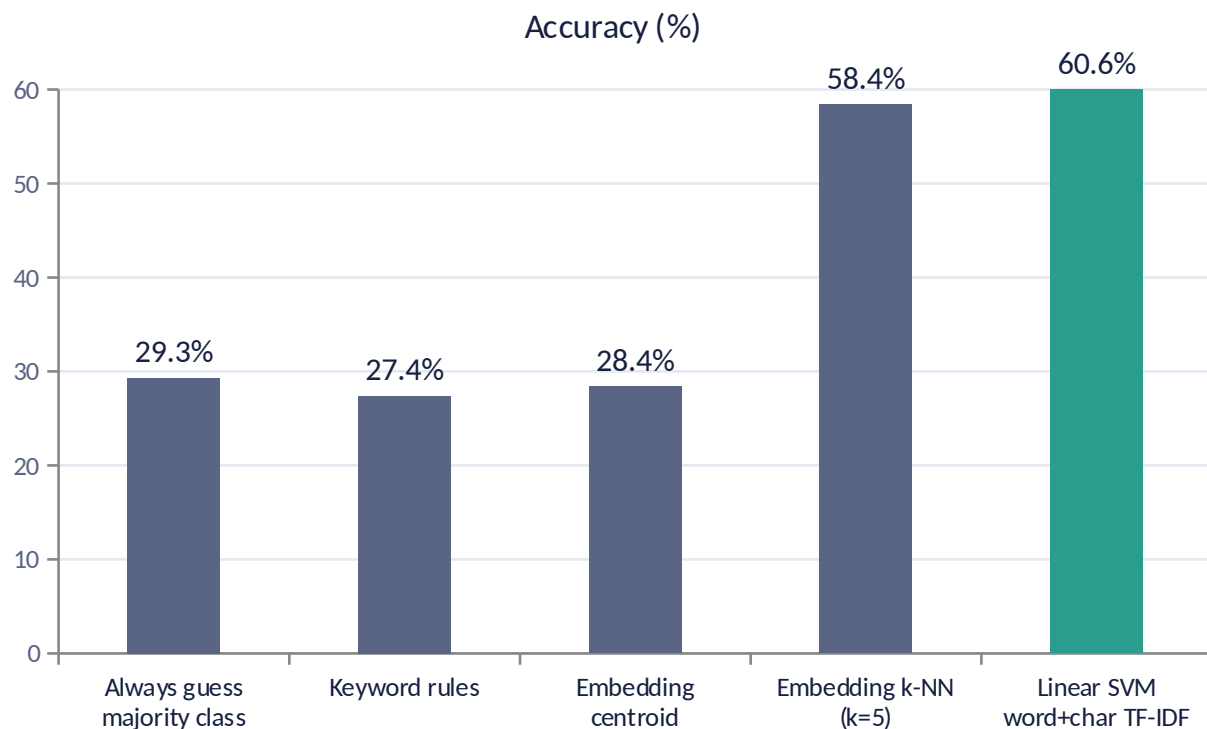
- multilingual MiniLM — a shared en/de vector space
- offline TF-IDF + SVD — no download, deterministic
- Identical width and normalisation, so the ES mapping never changes between them.

Guardrails, not prompts

- PII gate fails closed before any hosted call
- Invented queue labels rejected, not indexed
- RAG citations checked against retrieved ids

Result: routing tickets from text alone

Stratified 25% held-out split, $n = 7,145$



60.6%

best router accuracy

vs 29.3% majority

0.604

macro-F1

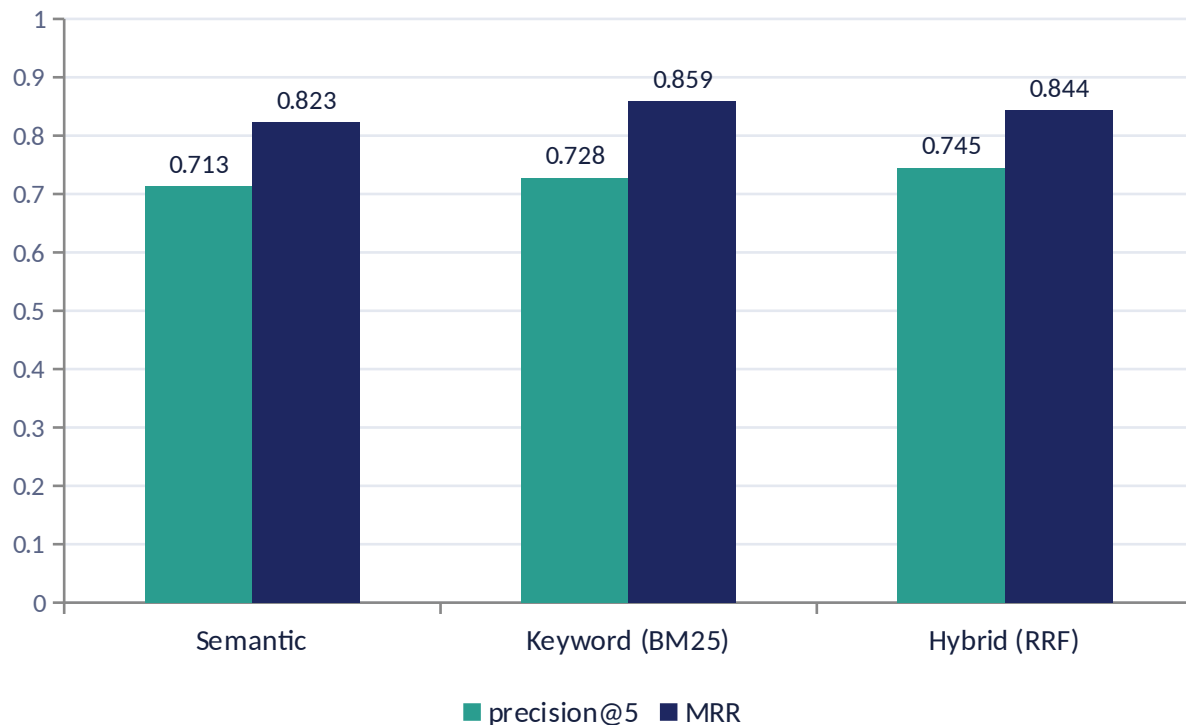
vs 0.045 majority

Hand-written rules lose to guessing

27.4% accuracy is below the 29.3% you get by answering "Technical Support" every time. The rules do carry signal — macro-F1 0.208 against 0.045 — but on raw accuracy they are worse than a constant. This is why we report both metrics.

The result that contradicted our design

250 queries, $k=5$, proxy relevance — a hit counts if it shares ≥ 2 tags with the query ticket



Semantic search did not beat BM25.

Why — and why it was predictable

These numbers come from the offline TF-IDF + SVD backend, i.e. LSA. LSA is a linear projection of the same term-document statistics BM25 already scores. It compresses lexical information; it adds no semantic information from outside the corpus.

The cross-lingual probe makes the point sharply: a German query retrieves ~0 relevant English tickets, because LSA shares no space across languages.

The multilingual neural model is expected to change both results. We have not measured it, so we do not claim it.

Routing is a different task: k -NN exploits the geometry of the space, so embeddings win there even in LSA form.

What the data told us

Findings that change what an operations team would do on Monday

The language label is broken in one direction

Tickets labelled de are wrong 27.2% of the time.

Tickets labelled en are wrong 0.05% of the time.

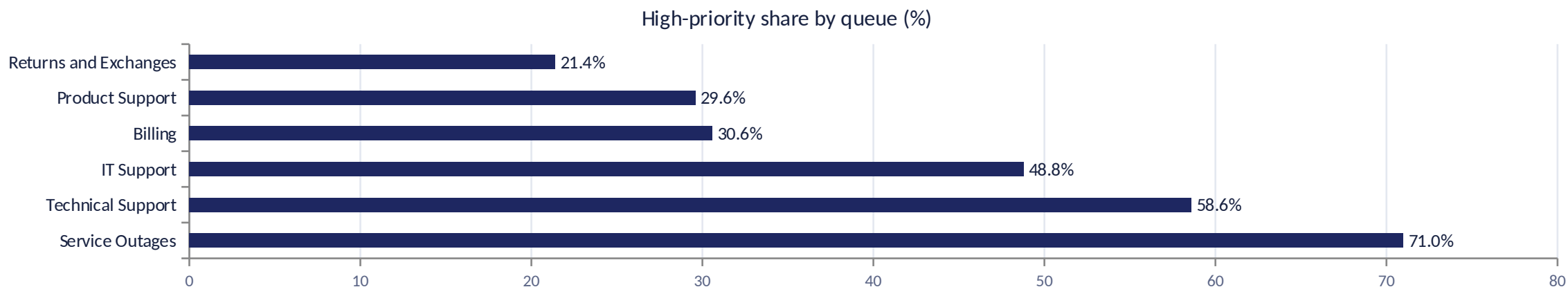
The 11.7% headline rate hides a one-directional defect: more than a quarter of the "German" corpus is English text.

Urgency is concentrated, not where volume is

Service Outages is 4.0% of volume but 71.0% high priority.

Human Resources is 9.5%.

Staffing to ticket volume would badly under-resource the outage queue.



Trade-offs we made deliberately

And the one we would change first

Pure-Python core, Spark as a thin wrapper

Every transformation is a pure function; UDFs call them. This is what makes 80 unit tests meaningful and lets the whole pipeline run without Spark. Cost: a second native implementation of the KPIs, cross-checked automatically.

At-least-once + idempotent sinks

Cheaper than transactional exactly-once, and the end state is identical because ticket_id is a content hash.

Exact k-NN offline, HNSW in Elasticsearch

At 28k documents brute force is fast and removes approximate-recall as a confound when comparing against BM25.

Macro-F1 reported alongside accuracy

With a 29.3% majority class, accuracy alone would have made the keyword rules look nearly as good as k-NN.

Weakest link

The proxy relevance metric. Shared tags reward topical similarity — right for routing and deduplication — but they under-credit a system that retrieves a solution phrased differently from the problem. Human judgements on a few hundred queries is the honest fix, and the first thing we would add.

Honest scope

80 unit tests pass and the pipeline ran end to end over all 28,587 rows. The Docker stack, the Spark jobs and the neural backend were not executed in the build environment — documented in the README.

Demo

GET /compare?q=...

One query, three rankings side by side — semantic, BM25 and hybrid. The clearest way to see what each retrieval mode actually does.

POST /route

Paste an unseen ticket. Returns the rule-based guess and the k-NN guess with its nearest neighbours, so the improvement is visible per request.

POST /ask

RAG over the corpus. Answers only from retrieved tickets; invented citations are stripped before you see them.

Kibana

KPI dashboards over the gold layer — queue volumes, sentiment, high-priority concentration.

Fallback: ``python -m tickets.offline_pipeline`` reproduces every number in this deck with no Docker, no API key and no network.

github.com/yaeldorin-glitch/biu-bigdata-support-tickets